

APIFlow

Subscription Billing & Usage-Metering Platform

TECHNICAL DOCUMENTATION

Laravel 12 · PHP 8.2+ · MySQL 8
Version 1.0 · September 2026

Table of Contents

1. Project Overview
2. Architecture
3. Tech Stack
4. Important Modules
5. Authentication Flow
6. Multi-Tenant Flow
7. Subscription / Plan Flow
8. Usage Metering Flow
9. Billing / Proration Flow
10. API Flow
11. Database & Relationship Overview

12. Important Business Rules
13. Validation & Restrictions
14. Customer / Admin Flow
15. Subscription Cancellation Flow
16. Usage Limit / Alert Flow
17. Important Implementation Details
18. Overall End-to-End Project Flow

1. Project Overview

APIFlow is a multi-tenant backend that meters customer API usage against a subscription plan and produces accurate, prorated billing.

1.1 What it does

- A **merchant** (tenant) defines one or more **plans** — name, base price, billing cycle, included usage units, overage rate per unit.
- **Customers** subscribe to a merchant's plan through a self-service portal.
- Every API call a customer makes is recorded as a **usage event**, aggregated daily.
- At the end of each billing cycle, the system generates an **invoice**: base price + overage beyond the included allowance, correctly prorated if the subscription started mid-cycle or the plan changed mid-cycle.

1.2 Who uses it

Role	Access
Merchant Admin	Manages plans, customers, subscriptions (view/change/cancel), invoices, and their own merchant dashboard.
Customer	Registers, subscribes, views usage/billing, changes plan, pays invoices — entirely self-service.

1.3 Demo tenants

Three demo merchants ship with the seed data, each with its own real (free, keyless) external API integration — this makes multi-tenancy something you can click through, not just take on faith:

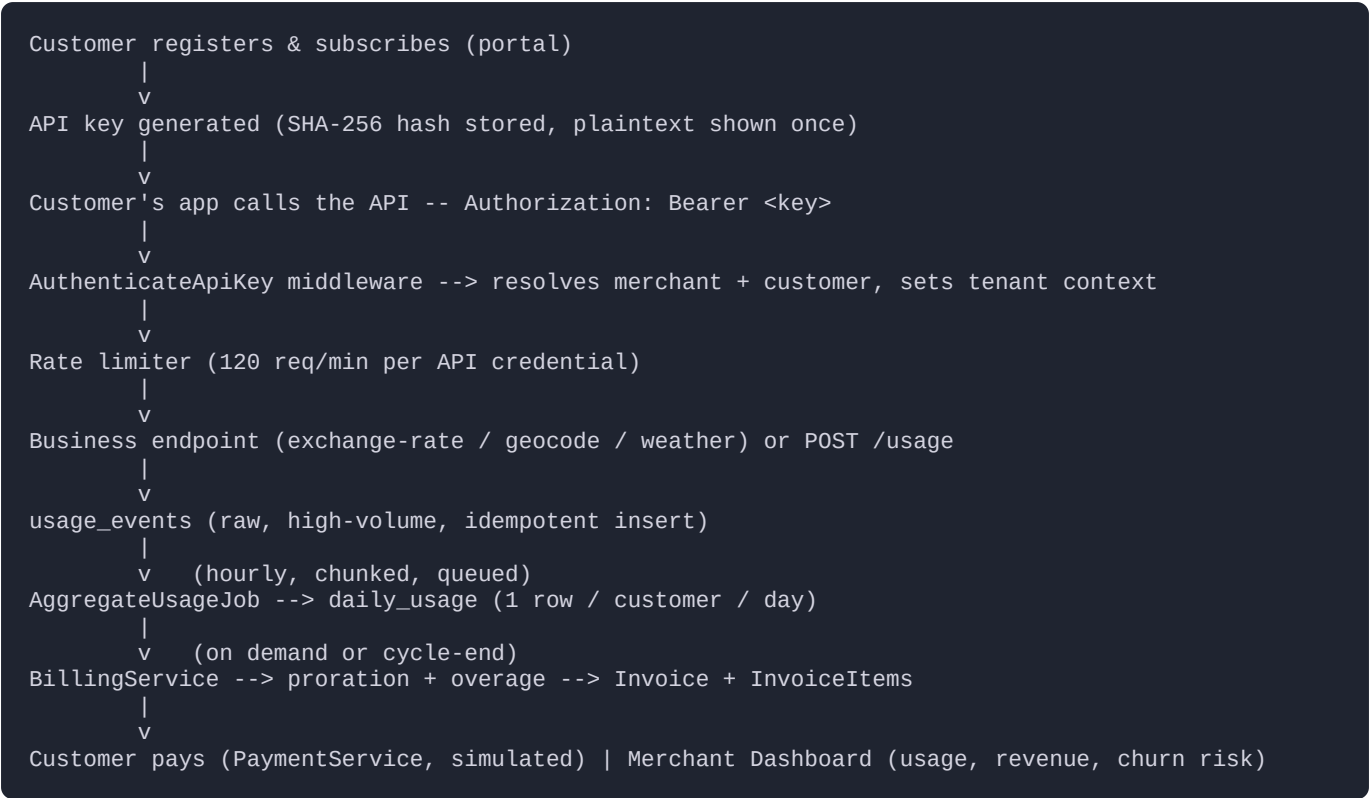
Merchant	Business	API
FinPay Technologies	Currency exchange	open.er-api.com
GeoLocate Pro	Geocoding	nominatim.openstreetmap.org
WeatherCloud	Weather forecasts	api.open-meteo.com

2. Architecture

2.1 Pattern

Standard Laravel MVC with a dedicated **service layer** — controllers stay thin (auth, validation, delegate, respond); all business logic (billing math, proration, tenant checks, caching) lives in single-purpose service classes. This keeps the same logic reusable across the admin UI, the customer portal, and the public API without duplication.

2.2 High-level request flow



2.3 Two separate applications, one codebase

	Merchant Admin	Customer Portal
Guard	<code>web</code> (App\Models\User)	<code>customer</code> (App\Models\Customer)
URL space	<code>/admin/...</code>	<code>/</code> (unprefixed)
Route file	<code>routes/admin.php</code>	<code>routes/customer-portal.php</code>
Login	<code>/admin/login</code>	<code>/login</code>

Each guard has its own auth middleware (`AuthenticateAdmin` / `RedirectIfAdminAuthenticated` , `AuthenticateCustomer` / `RedirectIfCustomerAuthenticated`) rather than Laravel's built-in `auth` / `guest` aliases — those hardcode a redirect to `route('login')` regardless of guard, which breaks the moment two login systems both need their own target.

2.4 Per-merchant branding

Each merchant carries `theme_from` / `theme_to` hex colors, applied as CSS custom properties (`--brand-from` / `--brand-to`) set inline per request — not Tailwind classes, since Tailwind only generates CSS for class names visible in source at build time, not colors read from the database at request time.

3. Tech Stack

Layer	Choice
Framework	Laravel 12, PHP 8.2+
Database	MySQL 8 (app data); SQLite in-memory (test suite)
Queue / Cache / Session	Laravel's <code>database</code> driver (Redis-ready — driver-agnostic code, swap via <code>.env</code>)
Auth	Hand-written session-based auth, two guards — no Sanctum/Breeze/Jetstream
Frontend	Blade + Tailwind CSS v4 (no React/Vue/Livewire, per assignment scope)
Admin list tables	Yajra DataTables — server-side search/sort/pagination against merchant-scoped SQL
PDF generation	barryvdh/laravel-dompdf — invoices, rendered identically for admin/API/portal
Testing	PHPUnit (Laravel's testing layer), 100 tests, unit + feature

4. Important Modules

4.1 Services (business logic)

Service	Responsibility
<code>SubscriptionService</code>	Subscribe, change plan, cancel (with final prorated billing), advance to next period
<code>BillingService</code>	Generates invoices — proration, overage, idempotency
<code>ProrationService</code>	Splits a billing period into per-plan segments around a mid-cycle change; prorates any amount by day-fraction
<code>UsageService</code>	Idempotent usage recording; demo usage simulation
<code>PlanService</code>	Create/update/delete plans with guardrails (seat limits, in-use protection)
<code>PlanPricingService</code>	Cached plan lookups (TTL + event-based invalidation)
<code>CustomerService</code>	Create/update/delete customers; portal password issuance; self-registration
<code>SubscriptionUsageSnapshot</code>	Single source of truth for "usage this cycle / overage / % used / 30-day trend" — shared by every customer-portal page that shows it
<code>DashboardService</code>	Merchant dashboard aggregates — top customers, projected overage revenue, churn risk, usage trend
<code>ApiCredentialService</code>	Issues/resolves/revokes API keys (SHA-256 hashed, prefix shown for identification)
<code>InvoicePdfService</code>	Renders an invoice to PDF — shared by admin, API, and portal download routes
<code>PaymentService</code>	Simulates a payment-gateway charge (stands in for Stripe) — real request/response shape, TODO marker for the real integration

4.2 Jobs & scheduled commands

Name	Type	Purpose
<code>AggregateUsageJob</code>	Queued job, hourly + on-demand	Rolls raw <code>usage_events</code> into <code>daily_usage</code> , chunked (5,000/batch), transactional, overlap-locked
<code>GenerateInvoiceJob</code>	Queued job	Generates one subscription's invoice for a specific (already-ended) period
<code>billing:generate-invoices</code>	Scheduled command, daily 01:00	Finds every active subscription whose period ended, dispatches its invoice job, advances it to the next period
<code>usage:aggregate</code>	Manual/scheduled command	CLI trigger for <code>AggregateUsageJob</code> (<code>--sync</code> to run inline for demos)

4.3 Middleware

- `AuthenticateAdmin` / `AuthenticateCustomer` — per-guard login enforcement
- `EnsureMerchantAccess` — blocks an admin from reaching another merchant's `{merchant}`-scoped routes
- `AuthenticateApiKey` — resolves the Bearer/X-API-Key credential, sets tenant context, rejects deactivated customers
- `throttle:api-credential` — 120 req/min per API key

5. Authentication Flow

5.1 Merchant Admin

1. Admin visits `/admin/login`, submits email/password.
2. `Auth::attempt()` against the `web` guard (`App\Models\User`).
3. Deactivated users (`is_active = false`) are logged straight back out with an error, even with correct credentials.
4. On success, redirected to `/admin/merchants/{merchant}/dashboard` for their own merchant.

5.2 Customer

1. Self-registration at `/register/{merchant:slug}` (or the `/register` landing page to pick a merchant first) — email + password + confirm only.
2. Or, a merchant admin issues/resets a portal password from the customer's admin page (plaintext shown once, only the hash stored).
3. Registration does *not* auto-login — it redirects to `/login` so the customer confirms the password actually works.
4. `Auth::guard('customer')->attempt()` — deactivated customers are rejected the same way as deactivated admins.
5. First login with no active subscription routes to `/plans/choose`.

5.3 API (machine-to-machine)

1. Admin generates an API key for a customer (`ApiCredentialService::generateKey()`) — plaintext shown once, SHA-256 hash stored, a short prefix kept for identification in the UI.
2. Caller sends `Authorization: Bearer <key>` (or `X-API-Key`).
3. `AuthenticateApiKey` middleware resolves the credential, checks the customer is active, sets tenant context for the rest of the request.
4. Invalid/revoked key ? `401` . Deactivated customer ? `403` .

6. Multi-Tenant Flow

6.1 Model: single database, shared schema

Every tenant-owned table carries a `merchant_id` column. Isolation is enforced in three layers (defense-in-depth, not just one):

Layer 1 — Global Eloquent Scope

`MerchantScope` automatically filters every query on a merchant-scoped model to the current tenant context. Applied via the `BelongsToMerchant` trait.

Layer 2 — Route Middleware

`EnsureMerchantAccess` blocks an admin from navigating to another merchant's `{merchant}`-prefixed routes at all.

Layer 3 — Explicit Controller Check

`VerifiesTenantOwnership` trait re-checks `merchant_id` explicitly inside the controller. This exists because of a real bug found during development: Laravel's `SubstituteBindings` resolves route-model-bound params (like `{plan}`) *before* `EnsureMerchantAccess` runs, regardless of declared middleware order — so a bound model can be resolved (unscoped) before tenant context even exists. Layer 3 closes that gap.

Lesson: never rely on a single isolation mechanism for multi-tenant data — middleware order is not guaranteed the way it looks in the route file.

6.2 Roles

One role today: `merchant_admin` — full access to plans, customers, subscriptions, and billing for their own merchant. A read-only "staff" tier existed earlier and was deliberately removed — the brief never asked for role differentiation, and it was pure branching logic with no real capability behind it. `UserRole` stays an enum (not a dropped column) in case a lighter tier is needed later.

There is no platform-wide super-admin — this is a single merchant's billing system, not a tool for onboarding tenants; the 3 demo merchants are seeded directly for that reason.

7. Subscription / Plan Flow

7.1 Plan attributes

- Name, base price (integer cents), currency
- Billing cycle — Monthly / Quarterly / Yearly
- Included units (usage allowance per cycle) and overage rate per unit beyond it
- Optional `max_subscribers` seat cap
- `is_active` — deactivating stops new signups without affecting existing subscribers

7.2 Subscribing

1. Customer picks a plan at `/plans/choose` (only their own merchant's active plans are offered).
2. `SubscriptionService::subscribe()` refuses if the customer already has an active subscription (one-at-a-time rule).
3. Period starts today, ends at the calendar boundary for that plan's billing cycle (see Section 9).

7.3 Changing plan (upgrade/downgrade)

1. Customer (their own subscription) or admin (any subscription in their merchant) can switch to another active plan.
2. Restricted to plans with the **same billing cycle** as the current one — see Section 12 for why.
3. A `subscription_plan_changes` row records old plan, new plan, and effective date (today).
4. Period dates are untouched — the next invoice splits the period into per-plan segments (Section 9).
5. Switching back to a previously-held plan is fully supported — there is no "one-way" restriction; each segment simply bills at whatever plan was active during it.

7.4 Subscription History

The customer portal's History page lists every subscription the customer has ever had (active, cancelled, or expired) plus the plan-change events within each — full audit trail, read-only.

8. Usage Metering Flow

8.1 Recording usage

1. Every API call — the generic `POST /usage` or a merchant's own business endpoint (exchange-rate/geocode/weather) — calls `UsageService::recordUsage()`.
2. Assumption: **1 API request = 1 usage unit**.
3. Insert is idempotent: unique `(merchant_id, event_key)` constraint + `insertOrIgnore`. The insert count (not a separate exists-check) tells the caller created-vs-duplicate, so concurrent retries can never double-count.
4. Row lands in `usage_events` — the raw, high-volume log — flagged `is_aggregated = false`.

8.2 Aggregation

1. `AggregateUsageJob` runs hourly (and on-demand via `usage:aggregate`).
2. Processes only `is_aggregated = false` rows, in chunks of 5,000 (`chunkById`, not offset-based `chunk()` — avoids the offset-scan cost past a few million rows).
3. Each chunk's totals update and `is_aggregated` flag commit together in one transaction — a mid-chunk crash rolls both back, so a rerun never double-counts.
4. A `WithoutOverlapping` job-lock means the hourly schedule and a manual run can never race each other.
5. Result lands in `daily_usage` — one row per customer per subscription per day. All billing and dashboard queries read from here, never from `usage_events` directly.

8.3 Merchant-gated business endpoints

Each demo merchant's own API (exchange-rate/geocode/weather) is gated to that merchant's own customers before the external call is even made — a customer of another merchant gets `403`. This is what makes the multi-tenant story concrete: three merchants don't just look different, each genuinely exposes a different real, working API.

9. Billing / Proration Flow

9.1 Calendar-aligned billing periods

A period runs to the end of the calendar month/quarter/year — not "one month from signup." An anniversary model would make a mid-cycle start structurally impossible to reason about, which the brief explicitly requires supporting.

9.2 Splitting a period around a plan change

`ProrationService::calculateSegments()` splits `[period_start, period_end]` into one segment per plan that was active during it, at each recorded change's effective date. No change ? one segment for the whole period.

9.3 The proration formula

```
prorated_amount = full_amount × (segment_days / total_cycle_days)
overage_units   = max(0, actual_usage_in_segment - prorated_included_units)
overage_cost    = overage_units × overage_rate_per_unit
```

total_cycle_days is the nominal full cycle length (e.g. all 30 days of the calendar month), not the segment's own length — dividing a short segment by itself would always give 100%, over-billing it. Both the base price *and* the included-units allowance are prorated the same way for each segment, so a customer only pays for, and only gets quota for, the time actually spent on each plan.

9.4 Worked example

Starter (?999/mo, 10,000 units, ?0.15/unit overage) ? Growth (?4,999/mo, 50,000 units, ?0.10/unit overage), switched on day 15 of a 30-day month:

Segment	Days	Base (prorated)	Included (prorated)
1–14 (Starter)	14	$999 \times 14/30 = ?466$	$10,000 \times 14/30 = 4,667$
15–30 (Growth)	16	$4,999 \times 16/30 = ?2,666$	$50,000 \times 16/30 = 26,667$

Each segment's actual usage is compared against *its own* prorated quota to compute that segment's overage. Both segments' base + overage are added into a single invoice with separate line items — nothing is "subtracted"; each segment is billed fairly for the plan that was active during it.

9.5 Invoice generation

- Triggered by `billing:generate-invoices` (daily, for any subscription whose period ended) or on-demand (admin's "Generate Invoice" button, or a subscription cancellation).
- Idempotent via `idempotency_key` (`sub_{id}_period_{start}_{end}`) — safe to call repeatedly for the same period.
- After billing, the subscription is advanced to its next period using its (possibly just-changed) current plan's cycle length.

10. API Flow

All API routes require `Authorization: Bearer <api_key>` (or `X-API-Key`), rate-limited to 120 req/min per key.

Method	Endpoint	Notes
GET	<code>/api/v1/exchange-rate</code>	FinPay only — real conversion via open.er-api.com
GET	<code>/api/v1/geocode</code>	GeoLocate Pro only — real lookup via OpenStreetMap Nominatim
GET	<code>/api/v1/weather</code>	WeatherCloud only — real current conditions via Open-Meteo
POST	<code>/api/v1/usage</code>	Generic usage event — every merchant's customers can use this one
GET	<code>/api/v1/invoices</code>	Lists the authenticated customer's own invoices
GET	<code>/api/v1/invoices/{invoice}/download</code>	Downloads that invoice as a PDF

10.1 Error handling

`/api/*` always renders errors as JSON, via `shouldRenderJsonWhen()`, rather than trusting the caller's `Accept` header — a misconfigured client never gets an HTML error page back.

10.2 Rate limiting

Keyed off the raw `Authorization / X-API-Key` header, not a request attribute set by earlier middleware — `ThrottleRequests` runs before custom middleware aliases regardless of route declaration order, so an attribute set by the auth middleware isn't available yet at that point.

11. Database & Relationship Overview

11.1 Tables

Table	Purpose	Key relationships
<code>merchants</code>	Tenants	has many plans, customers, users
<code>users</code>	Merchant admin logins	belongs to merchant
<code>plans</code>	Pricing/allowance definitions	belongs to merchant; has many subscriptions
<code>customers</code>	Merchant's end customers	belongs to merchant; has many subscriptions, invoices, <code>api_credentials</code>
<code>subscriptions</code>	A customer's plan enrollment	belongs to customer & plan; has many <code>plan_changes</code> , invoices
<code>subscription_plan_changes</code>	Audit trail of plan switches	belongs to subscription; references old/new plan
<code>api_credentials</code>	Hashed API keys	belongs to customer
<code>usage_events</code>	Raw per-call usage log (high volume)	belongs to customer & subscription
<code>daily_usage</code>	Aggregated daily totals	belongs to customer & subscription
<code>invoices</code>	Generated bills	belongs to customer & subscription; has many <code>invoice_items</code>
<code>invoice_items</code>	Line items (base charge / overage / proration)	belongs to invoice

11.2 Scaling `usage_events` to 50L+ (5M+) rows

- Composite indexes for the query patterns that matter: `(merchant_id, customer_id, recorded_date, is_aggregated)` for the aggregation sweep, `(subscription_id, recorded_date)` for billing lookups, `(created_at)` for future partitioning.
- `is_aggregated` flag means a rerun only ever touches unprocessed rows.
- `chunkById(5000)` instead of offset-based `chunk()` / `get()`.
- **Beyond ~10M rows:** partition by `recorded_date` (monthly RANGE), archive old partitions, read-replica for dashboard queries. Trade-off: a partitioned table's unique key must include the partition column, so idempotency would become per-calendar-day instead of forever — acceptable since event keys aren't reused across days in practice.
- `daily_usage` stays small (one row per customer per day) — every billing/dashboard query reads from here, never from `usage_events` directly.

11.3 Money

Every amount is an integer cents column — never float/decimal, to avoid rounding errors in billing math.

12. Important Business Rules

Enforced in the service layer so the system can never drift into an inconsistent state:

1. **Plan seat limit + 90% alert** — a plan can optionally set `max_subscribers` ; `Plan::isNearSubscriberLimit()` flags it once active subscriptions reach 90%, shown as a banner on the admin Plans page.
2. **Plan edits can't strand existing subscribers** — lowering `max_subscribers` below the plan's current active subscriber count is rejected.
3. **Plans with any subscription (active or past) can't be deleted** — so billing history/invoice line items never point at a deleted plan. Deactivate instead, which only blocks new signups.
4. **Deleting a customer auto-cancels their active subscription first** (billing a final invoice, see Section 15), then deletes; if cancellation fails, the delete is refused rather than leaving an orphaned subscription.
5. **Deactivating a customer** blocks portal login and API key use immediately, without touching their subscription record — reactivating restores access exactly as it was.
6. **Plan changes are restricted to the same billing interval** — switching monthly?quarterly mid-cycle isn't supported, since proration assumes the period's own cycle length throughout. Both the plan picker and server-side validation only offer/allow same- `billing_cycle` plans; a full cycle-length change takes effect at the next renewal instead.
7. **A customer has at most one active subscription at a time** — a plan change updates the existing subscription rather than creating a second one.

13. Validation & Restrictions

- **Merchant-scoped uniqueness** — a customer's email only has to be unique *per merchant*, not globally; the same person can be a customer of two different merchants.
- **Plan-change target validation** — the target plan must belong to the same merchant, be active, and share the current plan's billing cycle (Section 12.6). Enforced both in the UI (dropdown only offers valid options) and server-side (Form Request rule), so tampering with the request can't bypass it.
- **Cross-tenant mutation is blocked at three layers** — see Section 6.1.
- **Deactivated-customer requests are rejected at the middleware layer** — before any controller logic runs, for both portal login and API key auth.
- **Idempotency is enforced at the database level**, not just in application logic — unique constraints on `(merchant_id, event_key)` for usage, and on the invoice `idempotency_key`, so even a race between two concurrent requests can't create a duplicate.

14. Customer / Admin Flow

14.1 Admin capabilities

- Full create/edit/toggle access on Plans and Customers.
- Subscriptions are *admin-managed* but not *admin-created* — customers subscribe themselves via the portal. The admin side can view, **Change Plan**, **Cancel**, or trigger **Generate Invoice** on-demand.
- Invoices are read-only from the admin side — a record of what was billed, not something to edit after the fact.
- Merchant dashboard: current-cycle usage, top 5 customers, projected overage revenue, churn-risk customers (usage down >50% month-over-month), 30-day usage trend.

14.2 Customer capabilities (self-service portal)

Page	Purpose
Dashboard	Quick overview — active plan, usage this cycle, overage, 30-day trend, recent invoices
Subscription	Full plan/billing details; Change Plan (same-cycle only)
History	Every subscription ever held, plus plan-change events
Usage Details	Full usage breakdown + 90%/100% alerts + 30-day trend
Invoices	Full invoice list, status badges, download PDF, Pay Now
Profile	Account details, change password

Every customer-portal query is scoped to `customer_id = auth('customer')->id()` — a customer can never reach another customer's data, even by guessing an invoice ID (404, not 403, so ID enumeration can't distinguish "not yours" from "doesn't exist").

15. Subscription Cancellation Flow

Cancellation is currently **admin-initiated** (from the Subscriptions page) or happens automatically as a side effect of deleting a customer. There is no direct customer-facing "cancel my subscription" button in the current build.

15.1 What happens on cancel

1. `SubscriptionService::cancel()` validates the subscription is currently active.
2. **Bills a final invoice** for the days actually used this cycle — `period_start` through `min(today, period_end)` — using the same proration engine as a normal cycle-end invoice.
3. Skips that final invoice if the current cycle was *already* invoiced (e.g. an admin generated it on-demand earlier) — a `whereDate()` check against existing invoices for the same period prevents ever double-billing an overlapping range.
4. Marks the subscription `Cancelled` with a `cancelled_at` timestamp.
5. Carries a `// TODO: Need to cancel subscription in Stripe` marker for when a real payment provider is wired in.

15.2 Why there's no refund logic

There is nothing to refund: the customer was only ever billed for days already used (Section 15.1, step 2) — never for the unused remainder of the cycle. This mirrors how usage-based metered billing works generally (AWS, Twilio): you're billed for what you consumed, not a flat advance fee.

16. Usage Limit / Alert Flow

There are two distinct "limit" concepts in the system — easy to conflate, worth being precise about:

16.1 Plan seat limit (merchant-facing)

`max_subscribers` caps how many active subscriptions a *plan* can carry.

`Plan::isNearSubscriberLimit()` returns true at 90% of that cap — shown as a banner on the admin Plans page. This is optional per plan; `null` means unlimited.

16.2 Usage allowance (customer-facing)

Each subscription has an included-units allowance from its plan. `SubscriptionUsageSnapshot` computes, for the current cycle:

- `usedUnits` — sum of `daily_usage.total_units` within the current period
- `overageUnits` — `max(0, usedUnits - included_units)`
- `percentage` — `round(usedUnits / included_units * 100)`, capped at 100

This single service backs the Dashboard, Subscription, and Usage Details pages — they can never disagree on the numbers. At **90%**, an amber warning banner appears; at **100%**, a red banner explains that further usage is now billed at the overage rate.

17. Important Implementation Details

1. **Idempotency, two mechanisms** — usage events via a unique constraint + `insertOrIgnore` ; invoices via an `idempotency_key` string. Both make the operation safe to retry without a separate exists-check race.
2. **Calendar-aligned billing periods**, not anniversary-based — required for a mid-cycle-start scenario to even make sense.
3. **Proration divides by the nominal full cycle length**, not the billed period's own length — otherwise a short period always computes as 100% of itself.
4. **Tenant isolation has defense-in-depth** beyond the global scope — see Section 6.1's `SubstituteBindings` ordering story.
5. **Cache invalidation is TTL + event-based, not either alone** — `PlanPricingService` caches for 10 minutes; a model observer clears it immediately on update/delete, so staleness in practice is bounded by the observer, not the TTL.
6. **Rate limiting is keyed off the raw request header**, not a request attribute — see Section 10.2.
7. `/api/*` **always renders JSON errors**, regardless of the caller's `Accept` header.
8. **A date-cast column can round-trip with a time component on some database drivers** (observed on SQLite) — an exact string comparison against `toDateString()` can silently never match. Fixed by using `whereDate()` everywhere a date-only comparison is needed, which normalizes across drivers.
9. **Seed data lives in one place** (`DemoData::merchants()`) — every seeder loops over it instead of hardcoding a merchant lookup, so a 4th demo merchant is a data change, not a code change.

18. Overall End-to-End Project Flow

1. Merchant admin logs in, creates one or more **plans** (price, cycle, included units, overage rate).
2. A customer registers (self-service or admin-created) and lands on **Choose a Plan**.
3. Customer subscribes — a **Subscription** is created with a calendar-aligned period.
4. Admin issues the customer an **API key**.
5. Customer's application calls the merchant's API (or the generic usage endpoint) — each call is recorded as a **usage event**, rate-limited and idempotent.
6. Hourly, **AggregateUsageJob** rolls raw events into daily totals.
7. Customer can check their usage/allowance/overage at any time on their **Dashboard** or **Usage Details** page — with 90%/100% alerts.
8. Mid-cycle, the customer (or admin) can **change plan** — recorded for later proration, restricted to the same billing interval.
9. At cycle end (or on-demand), **BillingService** generates an **invoice** — base price + overage, correctly split across any mid-cycle plan-change segments.
10. Customer views the invoice and pays it (**Pay Now**, simulated gateway) — status moves Pending ? Paid.
11. Subscription automatically rolls into its **next period**, and the cycle repeats.
12. If the subscription is cancelled at any point, a final invoice bills only the days actually used, and status becomes **Cancelled**.
13. Throughout, the merchant admin's **Dashboard** shows live usage, projected overage revenue, and churn-risk customers across their entire customer base.